PolyFlop8

Fetch Unit Integration Test Card

TS-ST-FETCH-001

Verification of PC + ROM + IR + Decoder Instruction Pipeline

Test Plan ID:	TS-ST-FETCH-001
Revision:	1.1
Date:	January 6, 2026
Test Stage:	Stage 3 - Fetch Unit Integration
Author:	Artem Horiunov
Status:	APPROVED
Coverage Target:	90% Sequence Coverage

Based on PolyFlop8 Test Architecture Overview v1.0 & ISTQB Requirements v1.0

Contents

1 TEST OVERVIEW

1.1 Purpose

This document defines the test cases for Stage 3: Fetch Unit Integration testing of the PolyFlop8 microcontroller. This stage verifies the instruction supply chain consisting of Program Counter (PC), Program ROM, Instruction Register (IR), and Instruction Decoder.

1.2 Scope

- Verification of PC address sequencing and control modes
- Verification of ROM instruction fetch and timing
- Verification of IR latching and stability
- Verification of Decoder field extraction and parsing
- Control flow verification (jumps, branches, returns)
- **Exclusions:** Execution unit operations, memory writes, I/O operations

1.3 Test Configuration

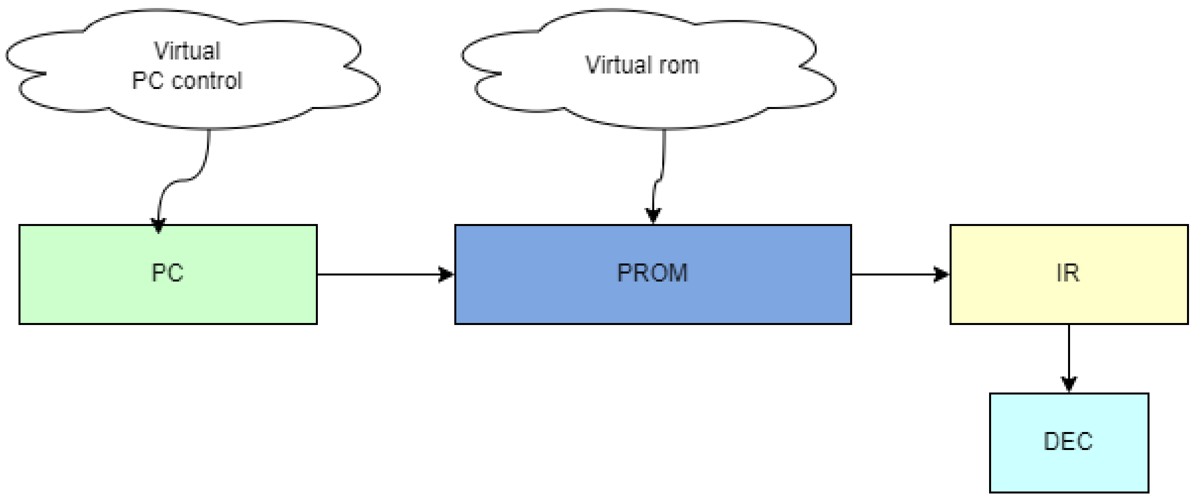

Figure 1: Fetch Unit Integration Test Configuration (from TS-ST-Overview-001)

Test Environment:

- Virtual PC control sequencer
- Virtual ROM with programmable instruction content
- Bypassed execution units (ALU, Register File)
- Direct monitoring of pipeline stages

1.4 Key Interfaces Under Test

Interface	Test Focus	Covered Requirements
PC → ROM	Address sequencing and control	TR-PC-01 to TR-PC-05
ROM → IR	Instruction fetch timing	TR-ROM-01, TR-ROM-02
IR → Decoder	Instruction stability	TR-ROM-03, TR-ROM-04
Decoder → Control	Opcode/field extraction	TR-DEC-01 to TR-DEC-04
Control → PC	Next address selection	TR-CU-06, TR-ISA-13 to TR-ISA-17

1.5 Pipeline Stages Verification

Pipeline Stage	Tested Behavior	Critical Timing
PC Generation	Address sequencing, jumps, branches	Clock cycle 0-2ns
ROM Access	Instruction fetch, timing, wait states	Cycle 2-5ns
IR Latching	Instruction stability, enable timing	Rising edge +1ns
Decoding	Opcode extraction, field parsing	Combinational delay <2ns
Control Handoff	Decoded signals to execution unit	End of cycle validation

2 TEST CASES

2.1 Test Case FT-01: Sequential Fetching

Test ID	FT-01
Priority	P1
Title	Sequential Fetching (PC increment)
Objective	Verify PC increments by 1 each clock cycle in fetch mode
Requirements	TR-PC-01, TR-ROM-01
Pre-conditions	Reset active (rst='1'). ROM loaded with known instruction pattern (e.g., 0x0000 for NOP). Virtual Control Unit configured.
Test Steps	1. Release reset (rst='0') 2. Configure PC source: pc_src="00" (Fetch/Sequential mode) 3. Enable PC: pc_en='1' 4. Apply 5 clock pulses 5. Monitor pc_out and instruction_out each cycle 6. Verify PC sequence and corresponding instruction fetch
Expected Results	• PC sequence: 0x000 $\rightarrow$ 0x001 $\rightarrow$ 0x002 $\rightarrow$ 0x003 $\rightarrow$ 0x004 • instruction_out shows corresponding ROM contents at each address • IR latches instruction on each rising edge when ir_en='1' • Decoder outputs update with extracted fields
Pass Criteria	PC increments correctly and fetches corresponding instructions
Coverage Points	Sequential address generation, instruction fetch timing

2.2 Test Case FT-02: Absolute Jump (JMP)

Test ID	FT-02
Priority	P1
Title	Absolute Jump (JMP instruction verification)
Objective	Verify PC loads absolute 11-bit address for JMP instructions
Requirements	TR-PC-04, TR-DEC-04, TR-ROM-04, TR-ISA-13
Pre-conditions	PC = 0x005. ROM contains JMP instruction at 0x005 with target address 0x020 in Jump11bit field. Virtual Control Unit configured for Jump mode.
Test Steps	1. Fetch JMP instruction from ROM at address 0x005 2. Latch instruction in IR (ir_en='1') 3. Decoder extracts Jump11bit field (should be 0x020) 4. Configure PC source: pc_src="11" (Jump mode) 5. Apply clock pulse 6. Monitor pc_out 7. Verify next instruction fetch from new address
Expected Results	• Decoder outputs: Jump11bit = 0x020 • After step 5: pc_out = 0x020 • Next instruction fetch from ROM address 0x020 _out shows instruction at 0x020
Pass Criteria	PC correctly jumps to absolute address 0x020
Coverage Points	Jump address extraction, PC jump mode, decoder field parsing

2.3 Test Case FT-03: Relative Jump (RJMP)

Test ID	FT-03
Priority	P2
Title	Relative Jump (RJMP with negative offsets)
Objective	Verify PC calculates relative address with signed offset
Requirements	TR-PC-02, TR-ISA-13
Pre-conditions	PC = 0x010. ROM contains RJMP instruction at 0x010 with offset field = -2 (0xFE in 8-bit signed). Virtual Control Unit configured for Branch mode.
Test Steps	1. Fetch RJMP instruction from ROM at address 0x010 2. Decoder extracts offset field (RS_IMM = 0xFE) 3. ALU calculates sign-extended offset: 0xFFFE (16-bit signed -2) 4. Configure PC source: pc_src="01" (Branch/Relative mode) 5. Apply clock pulse 6. Monitor pc_out 7. Verify PC + offset calculation
Expected Results	• Decoder outputs: RS_IMM = 0xFE • Sign-extension result: 0xFFFE (-2 in 16-bit) • PC calculation: (0x010 + 1) - 2 = 0x00F • After step 5: pc_out = 0x00F • Next instruction fetch from ROM address 0x00F
Pass Criteria	PC correctly calculates relative jump to 0x00F
Coverage Points	Sign-extension logic, relative address calculation, negative offsets

2.4 Test Case FT-04: Branch NOT Taken

Test ID	FT-04
Priority	P2
Title	Branch NOT Taken (Fall-through logic)
Objective	Verify PC continues sequential execution when branch condition false
Requirements	TR-PC-01, TR-ISA-15
Pre-conditions	PC = 0x010. ROM contains conditional branch instruction at 0x010. Branch condition evaluates to FALSE (e.g., BRBS with Z=0). Virtual Control Unit configured.
Test Steps	1. Fetch branch instruction from ROM at address 0x010 2. Evaluate branch condition (simulate condition FALSE) 3. PC source defaults to "00" (Sequential mode) when condition false 4. Apply clock pulse 5. Monitor pc_out 6. Verify sequential execution continues
Expected Results	• Branch condition evaluation: FALSE • PC source remains in sequential mode (pc_src="00") • After step 4: pc_out = 0x011 (PC + 1) • Next instruction fetch from ROM address 0x011
Pass Criteria	PC increments normally when branch condition not met
Coverage Points	Conditional branch logic, fall-through behavior, PC source selection

2.5 Test Case FT-05: Reset Behavior

Test ID	FT-05
Priority	P3
Title	Reset Behavior (Vector 0x000 enforcement)
Objective	Verify PC resets to 0x000 and fetches NOP from reset vector
Requirements	TR-PC-01, TR-ROM-05
Pre-conditions	PC at random address 0x555. ROM initialized with instructions. Reset signal controlled by testbench.
Test Steps	1. Set PC to 0x555 through forced assignment or jump 2. Assert reset (rst='1') 3. Monitor pc_out and instruction_out 4. Hold reset for 2 clock cycles 5. Release reset (rst='0') 6. Monitor PC and instruction fetch 7. Verify reset vector behavior
Expected Results	• Immediately after step 2: pc_out = 0x000 (async reset) • instruction_out = 0x0000 (NOP instruction) • IR contains NOP (0x0000) • Decoder outputs all zeros • After reset release: PC continues from 0x000
Pass Criteria	PC correctly resets to 0x000 and fetches NOP
Coverage Points	Reset logic, PC initialization, ROM reset vector

3 ADDITIONAL TEST CASES

3.1 Test Case FT-06: Instruction Decoder Field Extraction

Test ID	FT-06
Priority	P1
Title	Instruction Decoder Field Extraction
Objective	Verify all decoder output fields correctly extract instruction bits
Requirements	TR-DEC-01, TR-DEC-02, TR-DEC-03, TR-DEC-04
Test Steps	Test each instruction format: 1. R-Type: Opcode[15:11], RD[10:8], Rs[7:5] 2. I-Type: Opcode[15:11], RD[10:8], Imm[7:0] 3. J-Type: Opcode[15:11], Address[10:0] 4. Verify field overlap handling (Jump11bit overlaps RD/RS_IMM)
Expected Results	• Correct field extraction for each instruction type • Opcode: bits 15-11 • RD: bits 10-8 (destination register) • RS_IMM: bits 7-0 (source/immediate) • Jump11bit: bits 10-0 (jump address) • No corruption between overlapping fields
Coverage Points	Decoder bit slicing, field extraction, instruction format parsing

3.2 Test Case FT-07: ROM Boundary Conditions

Test ID	FT-07
Priority	P2
Title	ROM Boundary Conditions
Objective	Verify ROM behavior at address boundaries
Requirements	TR-ROM-01, TR-ROM-05
Test Steps	1. Test lowest address: PC = 0x000 2. Test highest address: PC = 0x7FF (11-bit max) 3. Test wrap-around: Jump from 0x7FF to 0x000 4. Test undefined addresses (simulate glitch to 0x800) 5. Verify NOP return for invalid addresses
Expected Results	• Address 0x000: Returns valid instruction • Address 0x7FF: Returns valid instruction 0x800+: Returns 0x0000 (NOP) • No crashes or undefined behavior at boundaries
Coverage Points	ROM address decoding, boundary handling, default behavior

3.3 Test Case FT-08: Pipeline Timing Verification

Test ID	FT-08
Priority	P2
Title	Pipeline Timing Verification
Objective	Verify fetch pipeline timing constraints
Requirements	TR-ROM-02, TR-PC-05
Test Steps	Measure timing for each pipeline stage: 1. PC to ROM address propagation 2. ROM access time (combinational delay) 3. IR setup/hold times relative to clock 4. Decoder propagation delay 5. PC enable/disable timing (pc_en)
Expected Results	• Total fetch cycle 10ns at 100MHz • ROM access 5ns • Decoder delay 2ns • IR setup time 1ns, hold time 0.5ns • PC enable/disable meets timing constraints
Coverage Points	Pipeline timing, critical paths, setup/hold constraints

3.4 Test Case FT-09: Indirect/Return Mode

Test ID	FT-09
Priority	P2
Title	Indirect/Return Mode (RET instruction)
Objective	Verify PC loads address from memory for return instructions
Requirements	TR-PC-03, TR-ISA-17
Pre-conditions	PC ready for return. ram_data input contains return address 0x030. Virtual Control Unit configured for Return mode.
Test Steps	1. Configure PC source: pc_src="10" (Indirect/Ret mode) 2. Set ram_data input = 0x030 3. Apply clock pulse 4. Monitor pc_out 5. Verify instruction fetch from return address
Expected Results	• After step 3: pc_out = 0x030 • Next instruction fetch from ROM address 0x030 • Note: Only supports page 0 (8-bit addresses)
Coverage Points	Return address loading, indirect PC control, 8-bit address limitation

3.5 Test Case FT-10: PC Enable/Disable (Stall)

Test ID	FT-10
Priority	P3
Title	PC Enable/Disable (Stall control)
Objective	Verify pc_en signal correctly stalls PC when low
Requirements	TR-PC-05
Test Steps	1. Start with pc_en='1', PC incrementing normally 2. Set pc_en='0' for 3 clock cycles 3. Monitor pc_out during stall 4. Set pc_en='1' again 5. Verify PC resumes from correct address 6. Test with different PC modes (sequential, jump, branch)
Expected Results	• During pc_en='0': PC remains unchanged • No instruction fetch updates during stall • After pc_en='1': PC resumes normal operation • Stall works in all PC modes
Coverage Points	PC stall logic, enable/disable timing, multi-mode stall

4 TEST EXECUTION PROCEDURE

4.1 Setup Instructions

1. Load Fetch Unit testbench (tb_fetch.vhd)
2. Configure virtual PC control sequencer
3. Load ROM with test program pattern (see Appendix A)
4. Initialize PC to known state (typically 0x000 after reset)
5. Configure clock generator: 100MHz, 50% duty cycle
6. Apply and release reset (active high, 2 clock cycles minimum)
7. Configure instruction monitor for all pipeline stages

4.2 Execution Order

Order	Test Case	Dependencies	
1	FT-05	None (reset establishes known state)	
2	FT-01	FT-05 (requires reset verification)	
3	FT-06	FT-01 (requires instruction fetch)	
4	FT-02	FT-01 (requires sequential fetch baseline)	
5	FT-03	FT-02 (requires jump verification)	
6	FT-04	FT-01, FT-02 (requires both fetch and jump)	
7	FT-09	FT-01 (requires PC control)	
8	FT-10	All PC modes (requires all PC operations)	
9	FT-07	All address modes (boundary testing)	
10	FT-08	All previous (timing verification last)	

4.3 Pass/Fail Criteria per Test

- **Pass:** All expected results match within timing constraints
- **Fail:** Any functional mismatch or timing violation
- **Warning:** Minor timing margin (<10% violation) with correct function
- **Blocked:** Test cannot execute due to dependency failure

4.4 Test Program for ROM

Listing 1: Sample Test Program for ROM Verification

```
-- ROM Memory Map for Fetch Unit Tests
-- Address  Instruction        Description
-- 0x000:   0x0000            ; NOP (Reset vector)
-- 0x001:   0x1234            ; Test pattern 1
-- 0x002:   0x5678            ; Test pattern 2
-- 0x003:   0x9ABC            ; Test pattern 3
-- 0x004:   0xDEF0            ; Test pattern 4
```

```
-- 0x005:   0xC020              ; JMP 0x020 (opcode=C, address=0x020)
-- 0x006:   0x0000              ; NOP (should not be reached)
-- ...
-- 0x010:   0xD0FE              ; RJMP -2 (opcode=D, offset=0xFE)
-- 0x011:   0x0000              ; NOP (fall-through target)
-- 0x00F:   0x5555              ; RJMP target address
-- ...
-- 0x020:   0x1111              ; JMP target address
-- 0x7FF:   0xFFFF              ; Highest address test
```

5 COVERAGE METRICS

5.1 Functional Coverage

Coverage Point	Description	Target
PC Modes	All 4 pc_src modes (00, 01, 10, 11)	100%
Address Ranges	PC addresses across full 11-bit range	100%
Instruction Formats	R-Type, I-Type, J-Type decoding	100%
Decoder Fields	All extracted fields (opcode, RD, RS_IMM, Jump11bit)	100%
Control Flow	Sequential, jump, branch taken/not, return	100%
Reset/Stall	Reset behavior and PC stall (pc_en)	100%
Boundary Conditions	Address boundaries, wrap-around, invalid addresses	100%
Pipeline Stages	All fetch pipeline stages exercised	100%

5.2 Code Coverage Targets

- **Statement Coverage:** 95%
- **Branch Coverage:** 90%
- **Condition Coverage:** 85%
- **Sequence Coverage:** 90% (primary target for this stage)
- **FSM Coverage:** 100% state/transition coverage for Control Unit FSM

6 SIGNAL MONITORING

6.1 Critical Signals to Monitor

Signal	Width	Monitoring Purpose
pc_out	11-bit	PC address verification
pc_src	2-bit	PC mode selection verification
pc_en	1-bit	PC stall control verification
prom_data	16-bit	ROM output verification
ir_out	16-bit	Instruction register content
ir_en	1-bit	IR write enable timing
opcode	5-bit	Decoded opcode field
RD	3-bit	Destination register field
RS_IMM	8-bit	Source/immediate field
Jump11bit	11-bit	Jump address field
alu_out	8-bit	Branch offset (for relative jumps)
ram_data	8-bit	Return address (for RET)
jump_abs_11bit	11-bit	Absolute jump address

6.2 Timing Checks

- PC to ROM address propagation: 2ns
- ROM access time: 5ns
- IR setup time: 1ns before clock edge
- IR hold time: 0.5ns after clock edge
- Decoder propagation: 2ns
- Total fetch cycle: 10ns at 100MHz
- Reset recovery: PC stable within 1 cycle after reset release

6.3 Pipeline Stage Monitoring

Pipeline Stage	Key Checkpoints
PC Generation	Address valid at clock edge, mode transitions
ROM Access	Data stable after access time, address decoding
IR Latching	Latch on rising edge with ir_en, hold through cycle
Decoding	Fields extracted correctly, combinatorial response
Control Handoff	Decoded signals stable for execution unit

APPENDIX A: TEST PROGRAMS AND VECTORS

6.4 Complete Test Program for ROM

Listing 2: Complete ROM Test Program

```
-- PolyFlop8 Fetch Unit Test Program
-- Assembled instruction codes for ROM

.org 0x000
  nop              ; 0x0000 - Reset vector
  ldi r1, 0x55     ; 0x1555 - Test immediate load
  ldi r2, 0xAA     ; 0x2AAA
  add r1, r2       ; 0x0120 - Test R-type format
  nop              ; 0x0000
  jmp label1       ; 0xC020 - Absolute jump to 0x020

.org 0x010
  rjmp -2          ; 0xD0FE - Relative jump back 2
  nop              ; 0x0000 - Should not execute
label2:
  ldi r3, 0xFF     ; 0x3FFF - Target of RJMP

.org 0x020
label1:
  lds r4, 0x40     ; 0x4440 - Direct load
  brbs 1, label3   ; 0xF101 - Branch if Z=1 forward
  nop              ; 0x0000 - Fall-through
  rcall sub1       ; 0xE030 - Relative call

.org 0x030
sub1:
```

```
  ret              ; 0x0008 - Return (special opcode)

.org 0x040
label3:
  nop              ; 0x0000 - Branch target
  ; Fill remaining with NOPs
  .rept 0x7C0
  nop
  .endr

.org 0x7FF
  nop              ; 0x0000 - Highest address
```

6.5 Test Vectors for Instruction Decoding

Instruction	Encoding	Opcode	RD	RS_IMM	Jump11bit	
ADD R1, R2	0x0120	00000 (0)	001 (1)	01000000	00100000000	
LDI R3, 0xFF	0x3FFF	00011 (3)	011 (3)	11111111	01111111111	
JMP 0x020	0xC020	01100 (C)	000 (0)	00100000	00000100000	
RJMP -2	0xD0FE	01101 (D)	000 (0)	11111110	00011111110	
BRBS 1, label	0xF101	01111 (F)	001 (1)	00000001	00100000001	

APPENDIX B: REQUIREMENTS TRACEABILITY

Test Case	Req ID	Requirement Description
FT-01	TR-PC-01	Verify Fetch Mode: PC increments by 1 each cycle
FT-01	TR-ROM-01	Verify instruction fetch returns correct 16-bit word
FT-02	TR-PC-04	Verify Jump Mode: PC loads absolute address
FT-02	TR-DEC-04	Verify Jump11bit output extracts bits [10:0]
FT-02	TR-ROM-04	Verify JMP instruction correctly maps 11-bit address
FT-02	TR-ISA-13	Verify Unconditional Jump updates PC immediately
FT-03	TR-PC-02	Verify Branch Mode: PC updates to PC+1+sign_extend(alu_out)
FT-03	TR-ISA-13	Verify Unconditional Jump (RJMP)
FT-04	TR-PC-01	Verify Fetch Mode (fall-through)
FT-04	TR-ISA-15	Verify Conditional Branch proceeds to PC+1 if condition NOT met
FT-05	TR-PC-01	Verify Reset behavior (implied)
FT-05	TR-ROM-05	Verify undefined addresses return NOP-equivalent
FT-06	TR-DEC-01	Verify opcode output extracts bits [15:11]
FT-06	TR-DEC-02	Verify RD output extracts bits [10:8]
FT-06	TR-DEC-03	Verify RS_IMM output extracts bits [7:0]
FT-06	TR-DEC-04	Verify Jump11bit output extracts bits [10:0]
FT-07	TR-ROM-01	Verify instruction fetch at all addresses
FT-07	TR-ROM-05	Verify undefined addresses return NOP
FT-08	TR-ROM-02	Verify ROM read is asynchronous
FT-08	TR-PC-05	Verify pc_en signal stalls PC when low
FT-09	TR-PC-03	Verify Indirect/Ret Mode: PC loads address from ram_data
FT-09	TR-ISA-17	Verify Return pops value from Stack into PC
FT-10	TR-PC-05	Verify pc_en signal effectively stalls PC when low